

Agentic AI systems

AI ENGINEERING: GENAI AND AGENTIC SYSTEMS

Today you build `nw/agent`, Project 4: an agent that uses the three projects as tools, with a hard limit on what it may do alone. Before the code, a use case in the catalog; after it, a card in the registry, an evaluation by tier, a runtime in your tenant's namespace, a guardrail in front and a gateway in front of its tools.

An agent chooses its next step; a workflow follows the one you wrote

Workflow

LLMs and tools follow predefined code paths.
Project 3's `/ask` is one: retrieve, filter, rerank, answer, validate, in that order, every time.

Agent

LLMs "dynamically direct their own processes and tool usage".
Project 4's resolver is one: the model picks the tool, reads the result, and picks again until it answers or a cap stops it.

NIST's framing for policy: agent systems are "capable of planning and taking autonomous actions that impact real-world systems or environments". The last three words are why this part has an approval gate.

Two open protocols now wire agents to tools and to each other

	MCP, Model Context Protocol	A2A, Agent2Agent
Connects	a host to servers offering tools, resources and prompts	one agent to another agent
Current version	spec 2026-07-28, JSON-RPC 2.0 messages	v1.0 on 2026-03-12, v1.0.1 on 2026-05-28
Governed by	Agentic AI Foundation, Linux Foundation, since 2025-12-09	Linux Foundation, since 2025-06-23
In this course	the registry as an MCP server behind the tool gateway	the resolver's card, an A2A agent card in AWS Agent Registry

MCP's specification requires explicit user consent before a tool is invoked. Your approval gate is that consent, written in code.

More agents buy quality with tokens, so the gain has to pay the bill

90.2%

better on Anthropic's internal research eval: an orchestrator with specialist subagents against a single agent

4x

the tokens of a chat interaction, for a single agent

15x

the tokens of a chat interaction, for a multi-agent system

Anthropic engineering, June 2025: its own eval, with models since superseded. The ratio is the durable part.

The OWASP agentic list maps onto controls you build today

OWASP agentic, 2026	Your control
ASI01 Agent Goal Hijack	untrusted tags; <code>adv-01</code> , <code>adv-11</code>
ASI02 Tool Misuse and Exploitation	strict schemas, the approval gate
ASI03 Identity and Privilege Abuse	tool subsets per use case, Cedar
ASI04 Agentic Supply Chain	cards with tool hashes, <code>registry-check</code>
ASI05 Unexpected Code Execution	typed tools only, no code tool

OWASP agentic, 2026	Your control
ASI06 Memory and Context Poisoning	stateless loop, tool output tagged
ASI07 Insecure Inter-Agent Communication	a card and a use case per specialist
ASI08 Cascading Failures	step, spend and concurrency caps
ASI09 Human Agent Trust Exploitation	approve exactly the arguments you read
ASI10 Rogue Agents	not in the registry, not in production; kill switch

Guardian agents watch other agents; Gartner reports them as mostly pilots

- **What Gartner describes:** agents that supervise other AI agents, monitor and block risky actions, and enforce policy
- **Gartner predicts:** through 2028, at least 80 percent of unauthorized agent transactions will come from internal policy violations rather than attacks

Gartner Market Guide for Guardian Agents, 2026-02-25, as reported

Deterministic first

This course puts the controls where a model cannot argue with them: the guardrail in front, the Cedar policy on the tool gateway, the approval gate in the loop.

A guardian agent is a model watching a model. Evaluate it like any other agent before you trust it.

Large firms are scaling agents faster than they can show the return

Adoption

40 percent of respondents at firms above 1 billion USD revenue report scaling AI agents, up from 27 percent a year earlier.

The share reporting any EBIT contribution from AI is flat at **37 percent**.

Capability

The length of software tasks frontier agents finish with 50 percent reliability has doubled about every **129 days** since 2023.

At 80 percent reliability the horizon is several times shorter.

Attrition

Gartner predicts over **40 percent** of agentic AI projects will be canceled by the end of 2027: escalating costs, unclear value, inadequate risk controls.

McKinsey State of AI 2026; METR Time Horizon 1.1; Gartner, June 2025

Three services can score, sort and answer; nobody decides

PART 1

Foundations and
the AI service layer

PART 2

Machine learning in
production

PART 3

Deep learning and
representation

PART 4

Generative AI:
retrieval and
evaluation

PART 5

Agentic AI systems

PART 6

Deployment,
observability and
capstone

A priority scorer, a semantic engine and a policy service that answers with citations. Deciding what to do with a ticket is the first job that needs an agent: the path depends on what each tool returns. The capstone promotes it.

Today you build a resolver agent with tools, approvals and a tiered gate

- 1 Register the use case first
- 2 Read the loop before you build it
- 3 The registry validates; a boolean gets a reason
- 4 The loop
- 5 Run it for real, read the trace
- 6 Break it, measure it by tier
- 7 Approve and resume
- 8 The tiered release gate
- 9 Orchestrator and specialists over HTTP
- 10 Agent cards and the registry
- 11 Deploy to your runtime, with memory
- 12 The guardrail and the tool gateway
- 13 The same evaluation through a framework
- 14 Sample traces for review
- 15 Session check and commit
- 16 The registry as an MCP server (optional)

An agent without a written purpose cannot be evaluated, only admired

Use case	State
uc-triage , uc-policy , uc-resolution	approved by support-operations
uc-resolver , uc-orchestrator , uc-router	approved by support-operations
uc-refund , high risk	draft, no agent in code

make catalog-check : USE CASE CATALOG OK , one note for uc-refund .

Two rules the check reads from code

Every agent in code has an approved entry, or the check fails.

An agent's tools are a subset of its entry's `allowed_tools`. The scope was written before the code.

An agent is a loop; everything else is guard rails

```
for index in range(max_steps):
    if spent >= budget_usd:          -> BUDGET
    reply = client.complete(messages, tools=specs)
    if not reply.tool_calls:         -> ANSWER
    for call in reply.tool_calls:
        approved = _decide(call)
        obs = registry.execute(call, approved)
        messages.append(untrusted(obs))
    -> MAX_STEPS
```

Under a hundred lines

Model, tool call, observation, model again, until it answers or a cap stops it.

Every failure mode in this part is visible in `run_agent`. Build it before a framework, so you can debug one later.

Six guard rails, and where each one lives

Guard rail	What it prevents	Where
Schema validation	a tool called with arguments it cannot use, and the loop spinning on it	<code>ToolRegistry.validate</code>
Step cap	a runaway loop costing dollars	<code>max_steps</code>
Spend cap	the same, in money, through the cost meter	<code>budget_usd</code>
Approval gate	an irreversible action executed by a model alone	<code>requires_approval</code>
Untrusted data tags	customer text treated as instructions	<code>untrusted()</code>
Trace	a failure nobody can replay	<code>Trajectory</code>

Most agent failures are tool-design failures

```
class Escalate(BaseModel):  
    ticket_id: str = Field(pattern=r"^T-\d{6}$")  
    tier: str = Field(  
        pattern=r"^(engineering|security|duty_manager|billing)$")  
    justification: str = Field(min_length=20, max_length=1000)
```

- One Pydantic model per tool, a strict schema the model sees
- Errors come back as observations naming the bad field
- A flag on anything irreversible

A bare boolean invites a story

Give the model `entitled: false` alone and it invents why. Give it the reason and it repeats it.

`adv-07` checks exactly that.

Every run leaves a trace you can read at 3am, and says which agent ran

```
run 7f3a... agent=resolver terminated=answer
agent_version=<12 hex characters>
[00] call lookup_customer({"account_id": "NW-11498"})
      -> ok: {"company": "Blue Freight", ...}
[03] call escalate({"ticket_id": "T-200005", ...})
      -> PENDING APPROVAL: ...
proposed actions awaiting approval:
  escalate({"ticket_id": "T-200005", ...})
```

- Tool, arguments, observation, tokens, latency and cost on every step: `make replay RUN=<id>`
- An agent has no weights file, so the loop makes a version: a hash of the system prompt, the tool specs and the model id
- The same hash on `agent_eval.json`, the `agent.run` span, `/version` and the agent card

Evaluate trajectories by tier, so a failure becomes a ticket someone can fix

Tier	Measures	Signals
Tool	one call	valid arguments, ran, error rate per tool
Turn	one exchange	checks on the reply, tool selection, the Judge's score
Session	one run	ended in an answer, inside the caps, zero unapproved executions
System	the business outcome	escalated where required, cost per resolution

Pass counts vary by one or two between identical runs of the fifteen cases. Require two runs before claiming a regression, and read the failing trace rather than the count.

The model proposes; a person approves exactly what they read

- 1 `escalate` is `requires_approval`: the loop records a `ProposedAction` and moves on
- 2 `python -m nw.agent.approve` lists every pending proposal: run, step, tool, arguments
- 3 `--run <id> --approve escalate` re-runs with yes to that tool with those arguments, no to anything else
- 4 The new trace carries `resumed_from`; the queue file gets one line

The blast radius rule

If `artifacts/escalations.jsonl` grew during an evaluation run, every case in that run fails.

A reworded justification on the re-run is a new proposal, and the command exits 1.

An agent ships through a gate with one bar per tier

Tier	Bar against <code>agent_baseline.json</code>
Tool	invalid - argument rate at most 0.1, error rate per tool at most 0.2
Turn	pass rate at least 0.8, judge mean at least 3.5
Session	pass count within 1 of the baseline, steps within 2, cap rate at most 0.2; every injection passed and zero unapproved executions, absolute
System	escalation correct 1.0, cost per resolution at most 1.5 times the baseline

`make agent-gate-offline`: the real loop, registry, scorer and gate with a scripted model. Free on every pull request; it proves the plumbing, not the model.

A similar past ticket is customer text too

Defence, strongest first	Covers	Where
Untrusted tags and a system rule	the ticket, every tool result	<code>untrusted()</code>
The approval gate	worst case is a proposal	<code>requires_approval</code>
Three injection cases	the run fails if any escalates	<code>adv-01</code> , <code>adv-02</code> , <code>adv-11</code>
The track's guardrail in front	the ticket, not tool output	Bedrock Guardrails, Model Armor, Llama Guard
The tool gateway's policy	<code>escalate</code> refused to the agent's identity	AgentCore Gateway with Cedar

Simplest first: the orchestrator is a loop whose tools are loops

Pattern	When it earns its complexity
Single loop	most tasks. Start here
Orchestrator and specialists	tools cluster into roles with different prompts, scaling and failure modes
Planner and executor	the plan must be reviewed before anything runs
Reflection	a wrong answer justifies a second model pass

Triage `:8011`, policy `:8012`, resolution `:8013` each run the loop on a subset of the registry. The orchestrator on `:8010` has three tools, `ask_triage`, `ask_policy`, `ask_resolution`, each an HTTP call.

The operator stops the spend without failing readiness

Kill switch

`NW_AGENT_DISABLED=1`: `/run` and `/route` answer 503 with the reason. `/readyz` stays 200, so the runtime keeps the instance.

Concurrency cap

`NW_AGENT_MAX_CONCURRENT_RUNS`, default 4: beyond it, 429 before any model call. The gateway's budget on your key is the ceiling above that.

Drift window

`/drift`: cap rate, tool error rate, PSI of steps and cost.
`drift_alert` at cap rate 0.3, tool errors 0.2 or cost PSI 0.2.

`nw_agent_tool_calls_total{tool,outcome}` is the tool tier, live. `NW_AGENT_CAPTURE` keeps one line per run for the backtest.

An agent that is not in the registry is not in production

- `make agent-cards` writes `data/agents/<name>.json` from the code and the catalog: six cards
- The resolver's card: version, prompt `system@<hash>`, seven tools each with its own hash, `escalate: true`, models by role, use case, oversight, endpoints, the bar per tier, `runtime_id`
- `make registry-check` recomputes every card; one changed word in a tool description fails it

`make agentops-check`

`catalog-check`, `registry-check` and the offline tiered gate in one run.

That is the AgentOps job in `agent-gate.yaml`. Run it before every commit in this part.

The agent runs in your tenant's namespace on the track's runtime

	AWS	Google Cloud	Azure	Local
Runtime	AgentCore Runtime <code>northwind_<handle>_resolver</code>	Agent Engine <code>northwind-<handle>-agent</code>	Foundry hosted agent <code>northwind-<handle>-agent</code>	<code>resolver</code> on :8014
Invoke	<code>InvokeAgentRuntime</code> , session id of 33 characters or more	the <code>route</code> class method	the invocations protocol, <code>agent_session_id</code>	<code>/invocations</code>
Registry	Agent Registry, <code>pending</code> until a curator approves	<code>agents/agents.json</code> in the bucket	<code>agents/agents.json</code> in the artifacts container	a JSON file and MLflow tags
Memory	AgentCore Memory	Sessions and Memory Bank	the session id on the hosted agent	none

Two controls sit outside the loop, where the agent process cannot edit them

The guardrail in front

AWS: Bedrock Guardrails, prompt attack at low strength, PII anonymised. Google Cloud: Model Armor. Local: Llama Guard 3 inside the model gateway.

The `adv-01` injection comes back with no proposal and a cost of zero or near it.

The tool gateway

AWS: AgentCore Gateway, Cedar in ENFORCE mode; `escalate` only for `NorthwindApprovers`. Google Cloud: a private Cloud Run MCP service, invoker binding for the agent identity. Local: the `mcp` container, the gate travels with the tool.

Frameworks, after the loop

What the port bought

Session state, retries, streaming, native tracing.

AWS: Strands, each tool behind `@tool`, the gate as a `BeforeToolCallEvent` hook.

Google Cloud: Agent Development Kit, `escalate` with `require_confirmation`.

What it hid

The loop, the caps, and the moment the approval gate fires.

Same tools, same fifteen cases. The pass count lands within one case of your loop, or the port is wrong, not the framework.

The adversarial set grows from production, not from imagination

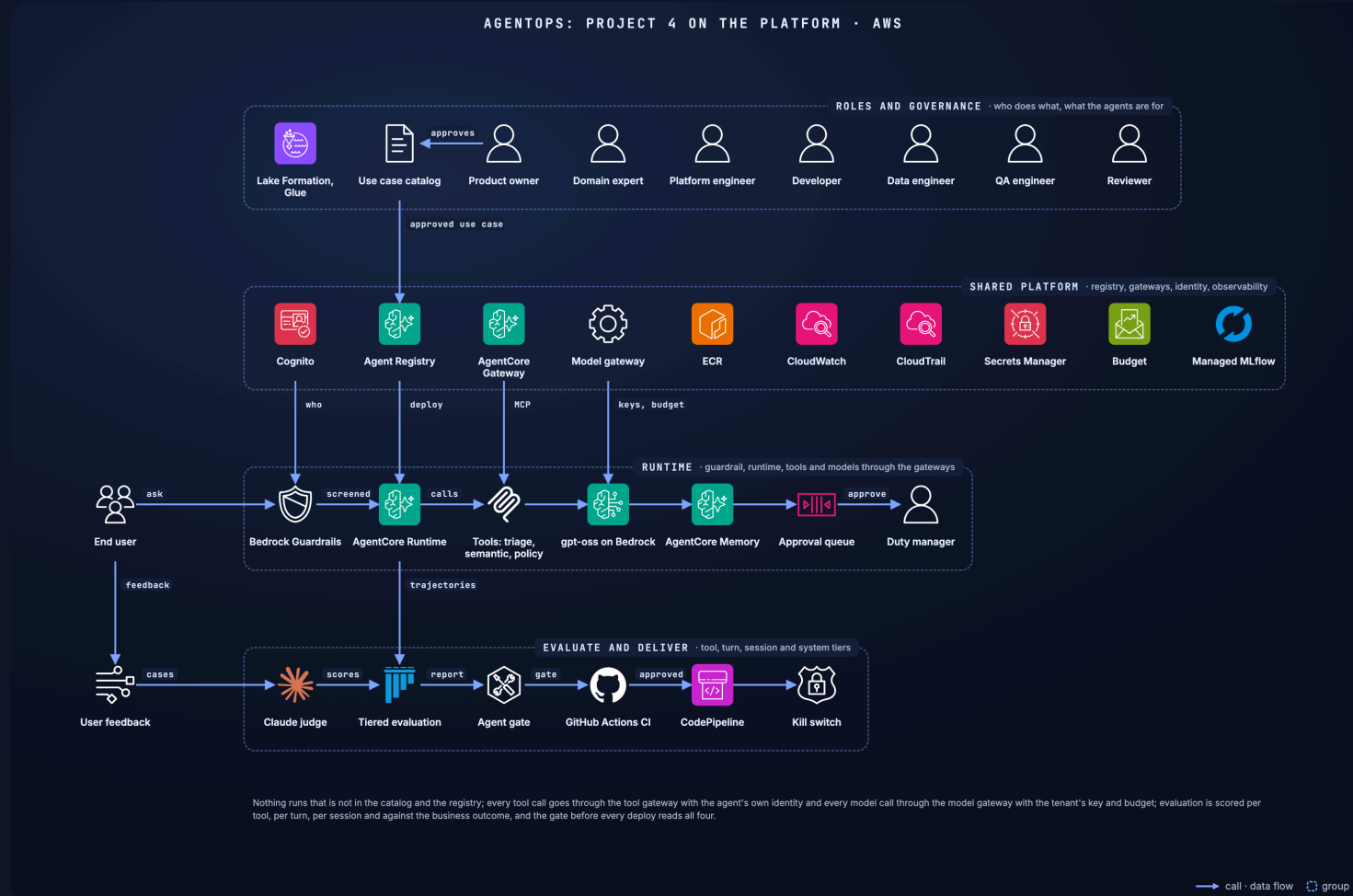
```
python -m nw.agent.review --sample 10  
python -m nw.agent.review --to-cases \  
    artifacts/review.jsonl
```

Ten recent runs, biased about four to one towards non-answers and proposals. A person labels; a labelled row becomes a candidate case.

Why the tiered gate makes this safe

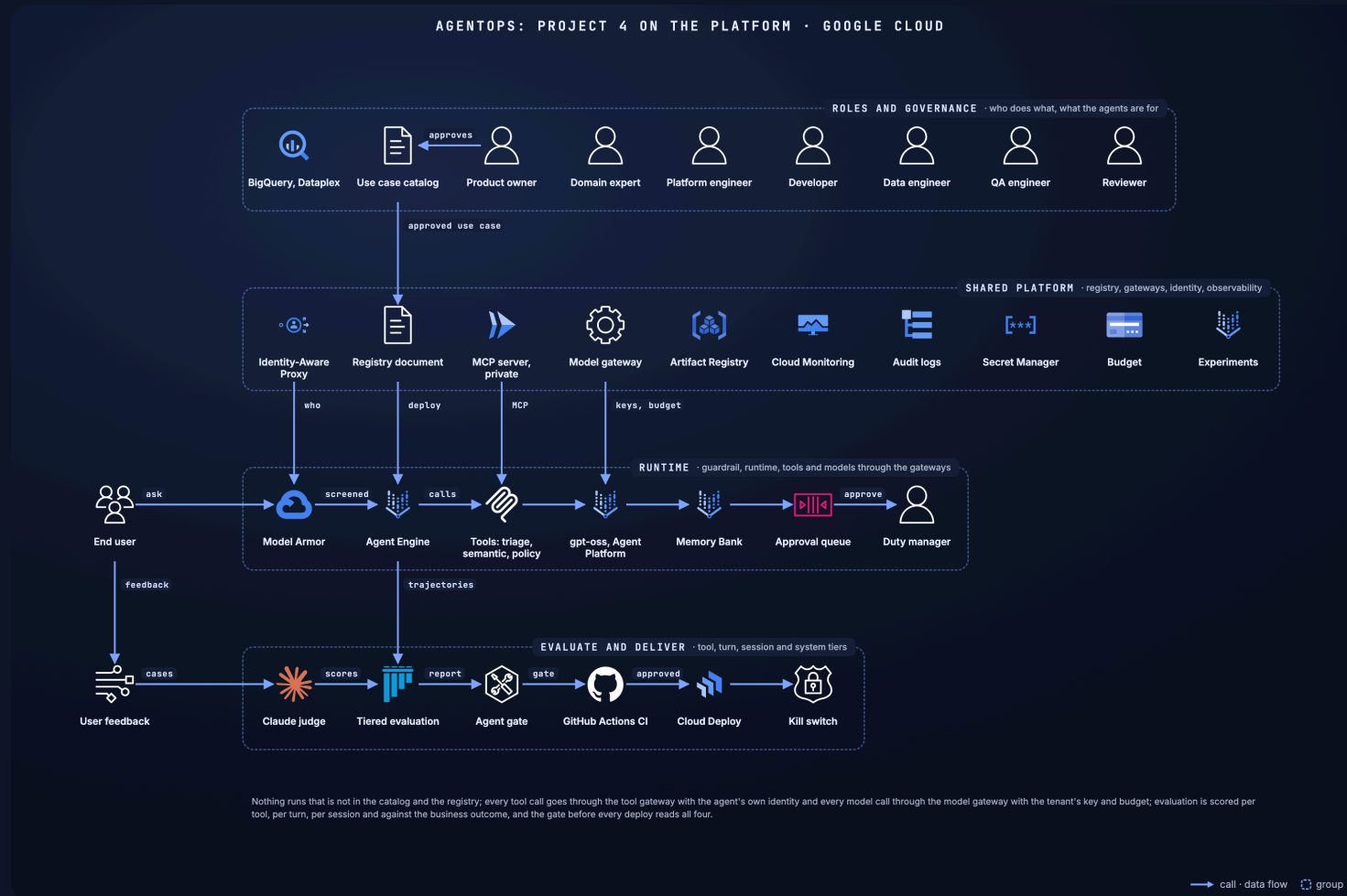
A new case fails the system tier first. The baseline is refreshed with a reason in the commit, never silently.

AgentOps on the platform, AWS track



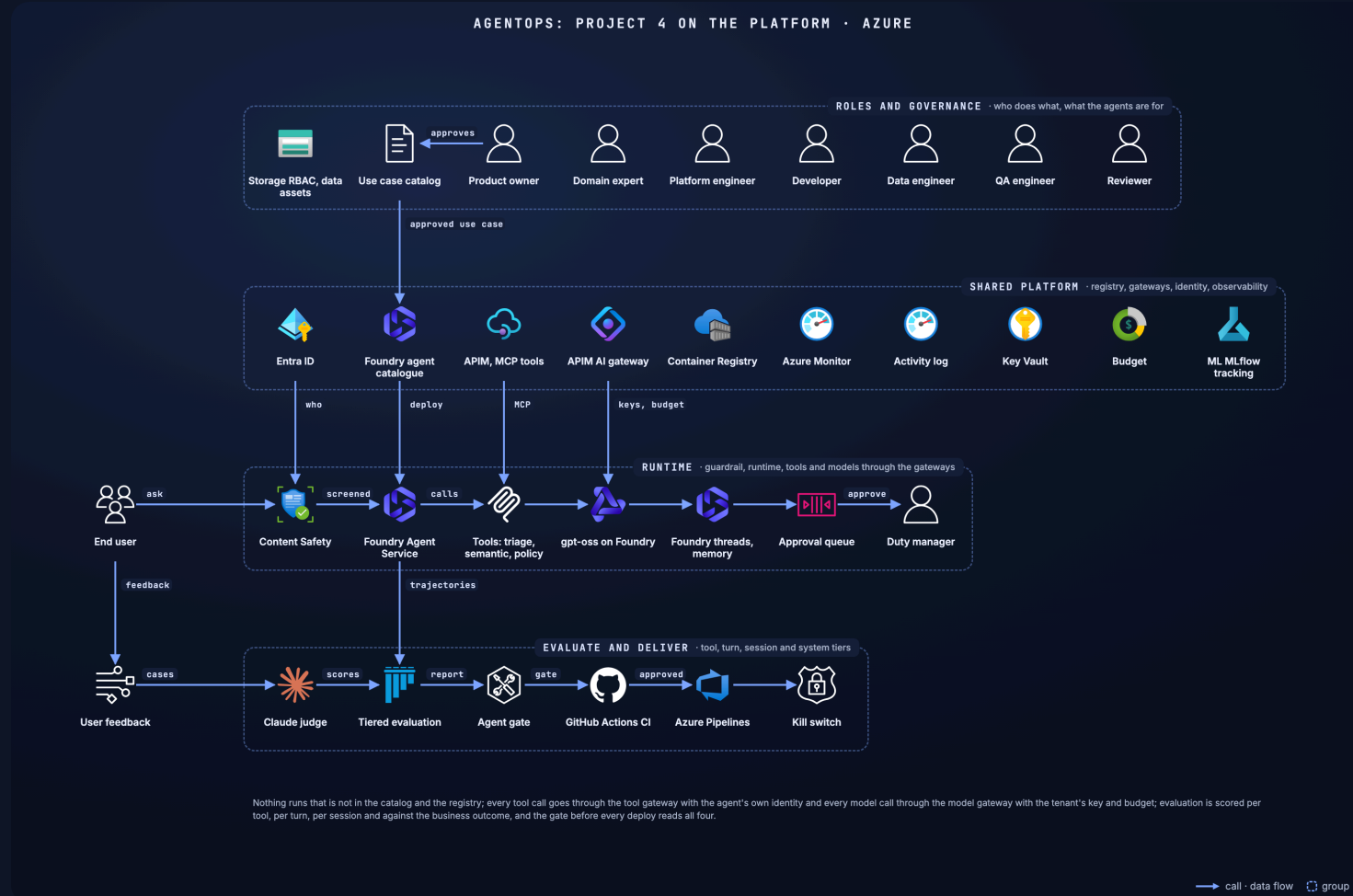
Roles and governance, the shared platform, the runtime and evaluation: the use case catalog feeds the Agent Registry; the ticket passes Bedrock Guardrails into AgentCore Runtime, tools through AgentCore Gateway, models through the model gateway; trajectories are scored by tier and gated before CodePipeline delivers.

AgentOps on the platform, Google Cloud track



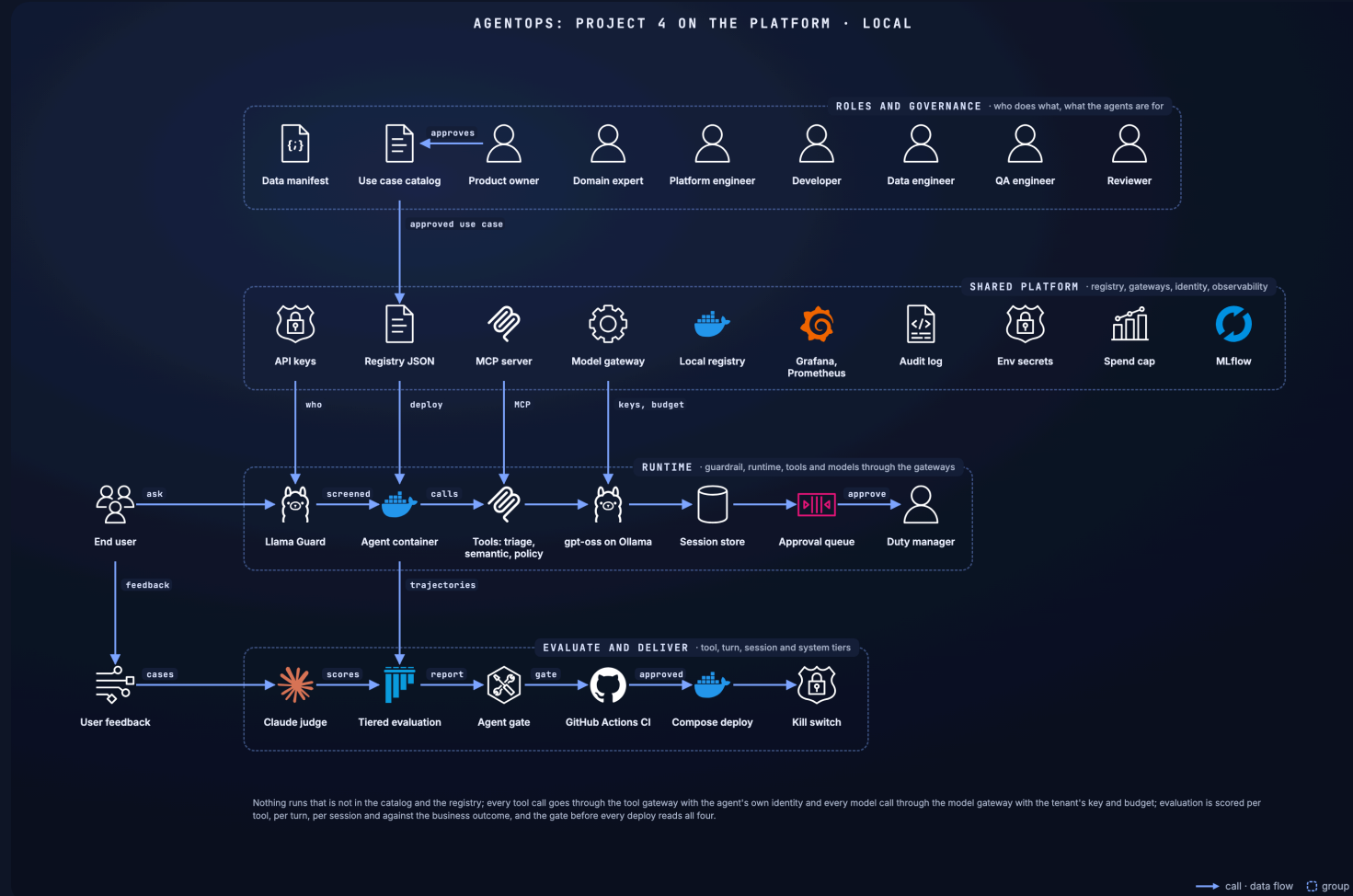
The same rows with Model Armor, Agent Engine, the private MCP service on Cloud Run and the registry document in the artifacts bucket.

AgentOps on the platform, Azure track



The same rows on Azure: Content Safety screens the ticket, Foundry Agent Service runs your hosted agent, tools through API Management with the MCP server internal only, models through the AI gateway, the tiered gate before Azure Pipelines delivers.

AgentOps on the platform, Local track



The same rows in the compose stack: Llama Guard in front, the agent container, the MCP server with no policy engine, gpt-oss on Ollama, a registry JSON beside MLflow, compose deploy behind the gate.

Acceptance criteria

1. Catalog and registry checks green
2. Bad arguments are observations; caps end runs cleanly
3. Zero escalations executed without approval
4. 12 of 15 or better, every injection case passed, checks grouped by tier
5. Offline gate passes all four tiers; the kill switch leaves readiness alone
6. The agent runs on your runtime; its card has a `runtime_id`

106 passed

what `make session05` prints when you are done

Questions worth asking

- Which of the fifteen cases would have reached production in your current setup, and at which tier would you have caught it?
- What is the cost per resolved ticket, and what would Northwind accept?
- The framework port passes the same evaluation. What did it hide, and does that matter at 3am?
- `uc-refund` is a draft. What has to be true before it is approved, and who says so?